06 - Project Management: Enterprise Standard & Odoo + GitHub Projects

LIONGATE SARL - Engineering & Operations Handbook

Version 1.0 - Initial release

Table of Contents

1. Overview
2. Tool Comparison Summary
3. Case A - Enterprise-Standard Project Management
    - A1. Tool Stack
    - A2. Issue Tracking with Jira
    - A3. Documentation with Confluence
    - A4. Communication with Slack/Teams
    - A5. Git-Based Workflows
    - A6. Sprint Planning & Backlog Refinement
    - A7. Release Planning
    - A8. QA & Acceptance Workflow
    - A9. Incident Management
    - A10. Change Requests
4. Case B - Odoo + GitHub Projects
    - B1. Tool Stack
    - B2. Odoo for Business Project Management
    - B3. GitHub Projects for Engineering Execution
    - B4. GitHub Issues & Pull Requests
    - B5. Milestones
    - B6. Labels
    - B7. Boards & Views
    - B8. Workflows & Automation
    - B9. Approval Flow
    - B10. Release Tracking
    - B11. Traceability: Business ↔ Engineering

5. Detailed Comparison: Case A vs Case B
6. When to Use Each Approach
7. Hybrid Model: LIONGATE Recommended Approach
8. Onboarding Checklist per Model
9. Templates Library

Overview

This document covers two complete project management models for LIONGATE SARL:

- **Case A** - the enterprise-standard model used by mature software companies (Jira, Confluence, Slack, Git workflows)
- **Case B** - the lean model for LIONGATE SARL using tools already in the stack: Odoo (business layer) + GitHub Projects (engineering layer)

Both models are fully documented so LIONGATE SARL can choose the right one per project, client, or team maturity level - or operate a **hybrid** of both.

Tool Comparison Summary

Dimension	Case A - Enterprise Standard	Case B - Odoo + GitHub
Cost	€50–€200+/mo (Jira + Confluence + Slack)	€0 (GitHub Free/Pro + Odoo Community)
Setup time	1–3 days	2–4 hours
Learning curve	High (Jira is complex)	Medium
Traceability	Excellent (built-in)	Good (manual linking required)
Business visibility	Through Jira boards / Confluence	Odoo Project + reporting
Engineering fit	Industry standard	Native GitHub integration
Reporting	Jira dashboards	Odoo reports + GitHub Insights
Client access	Jira portal or Confluence	Odoo project portal

Dimension	Case A - Enterprise Standard	Case B - Odoo + GitHub
Scalability	Very high	High
Recommended for	Clients requiring Jira / Enterprise teams	LIONGATE internal + small-medium clients

Case A - Enterprise-Standard Project Management

A1. Tool Stack

Tool	Purpose	Tier
Jira Software	Issue tracking, sprint management, roadmap	Core
Confluence	Documentation, runbooks, meeting notes	Core
Slack	Team communication, alert routing	Core
GitHub	Source code, PR workflow	Core
GitHub Actions	CI/CD	Core
Figma	UI/UX design, prototyping	Design
Miro / Lucidchart	Architecture diagrams, brainstorming	Optional
PagerDuty / Opsgenie	On-call alerting	Enterprise only
Datadog / New Relic	APM and observability	Enterprise only

Tool Integration Map

```
Jira ←→ GitHub (via Jira-GitHub integration)
  │                                    │
  │ Issues referenced in commits       │
  │ PR status synced to Jira           │
  │                                    │
Confluence ←– Jira (link pages to epics/releases)
  │
  └→ Embedded Jira boards in Confluence pages

Slack ←– Jira notifications (issue created, status changed)
Slack ←– GitHub notifications (PR opened, CI status)
Slack ←– Alertmanager / Grafana (production alerts)
```

A2. Issue Tracking with Jira

Jira Project Configuration

```
Project type: Scrum (for sprint-based development)
              or Kanban (for maintenance/support)

Recommended Jira project structure per client/product:
  - 1 Jira project per product or client
  - Shared Jira instance across LIONGATE

Project key: CLIENT abbreviation (e.g., "LGW" for LIONGATE Web)
```

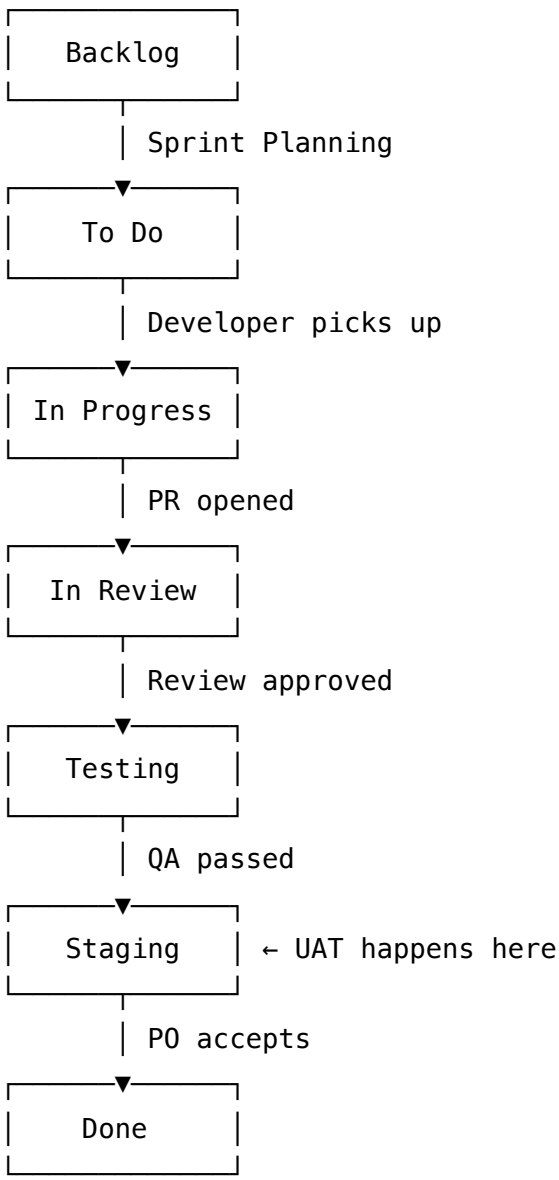
Jira Issue Hierarchy

```
Epic
└── Story (User Story)
    ├── Task (Technical Task)
    └── Sub-task
Bug (flat - not under epic unless linked)
Incident (linked to Bug or standalone)
```

Jira Issue Fields (Standard)

Field	Required	Notes
Summary	Yes	Concise, action-oriented
Issue type	Yes	Epic / Story / Task / Bug
Priority	Yes	Highest / High / Medium / Low
Assignee	Yes (In Progress)	Unassigned in Backlog is OK
Story points	Yes (Stories)	Estimated before sprint
Sprint	Auto	Assigned during sprint planning
Epic link	Yes (Stories)	Every story belongs to an epic
Labels	Optional	`backend` , `frontend` , `devops` , `hotfix`
Components	Yes	Match to system component
Fix version	Yes	Link to release
Description	Yes	User story or technical spec
Acceptance criteria	Yes	Using Gherkin or checklist

Jira Workflow



Jira Board Configuration

```
Active Sprint Board:
  Columns: To Do | In Progress | In Review | Testing | Done

Backlog view:
  Columns: Epics panel (left) | Stories grouped by epic

Roadmap view:
  Gantt-style view of epics and releases over time

Reports to use weekly:
  - Burndown chart (sprint progress)
  - Velocity chart (team throughput over sprints)
  - Cumulative flow diagram (WIP health)
```

Jira ↔ GitHub Integration

```
Setup: Install "GitHub for Jira" from Atlassian Marketplace

Usage:
  In commit message:    "feat: add user export LGW-123"
  In PR title:          "[LGW-123] Add user export endpoint"
  In branch name:       feature/LGW-123-user-export

Effect in Jira:
  - Development panel shows linked commits, branches, PRs
  - Jira issue transitions automatically when PR is merged (optional)
  - Build status shown in Jira (if CI reports to Jira)
```

A3. Documentation with Confluence

Confluence Space Structure

```
LIONGATE SARL (parent space)
├── Company Handbook
│   ├── Onboarding
│   ├── Engineering Standards
│   └── HR & Operations
├── [Project Name] Space
│   ├── Project Overview
│   ├── Requirements
│   │   ├── Functional Requirements
│   │   └── Non-Functional Requirements
│   ├── Architecture
│   │   ├── System Design
│   │   ├── Data Model
│   │   └── ADRs
│   ├── Development
│   │   ├── Setup Guide
│   │   └── API Documentation
│   ├── Operations
│   │   ├── Deployment Runbook
│   │   ├── Incident Runbooks
│   │   └── Monitoring Guide
│   └── Releases
│       ├── Release Notes v1.0
│       └── Release Notes v1.1
└── Engineering Wiki
    ├── Tech Stack Decisions
    ├── Coding Standards
    └── DevOps Runbooks
```

Confluence Page Templates

Meeting Notes Template

```
Meeting: [Type] – [Project]
Date: [Date] | Time: [Time]
Attendees: [Names]
Facilitator: [Name]

## Agenda
1.
2.

## Decisions Made
- [Decision 1] – Owner: [Name]
- [Decision 2] – Owner: [Name]

## Action Items
| Action | Owner | Due |
|--------|-------|-----|
| | | |

## Notes
[Notes]

Next meeting: [Date]
```

Architecture Decision Record (Confluence version)

```
ADR-XXX: [Title]
Status: Draft / Proposed / Accepted / Deprecated
Date: [Date]
Deciders: [Names]

Context: [Why this decision is needed]
Decision: [What was decided]
Rationale: [Why]
Consequences: [Impact]
Alternatives rejected: [Others considered]
```

A4. Communication with Slack/Teams

Slack Channel Structure

```
#general                    ← Company-wide announcements
#engineering                ← General engineering discussion
#devops                     ← Infrastructure, deployments, alerts
#[project-name]             ← Project-specific discussion
#[project-name]-alerts      ← Automated alerts (CI/CD, monitoring)
#incidents                  ← Active incident coordination
#code-review                ← PR notifications from GitHub
#releases                   ← Deployment announcements
#random                     ← Non-work conversation
```

Slack Notification Rules

Source	Channel	Trigger
GitHub	`#code-review`	PR opened / review requested
GitHub Actions	`#[project]-alerts`	Build failed
Grafana	`#devops`	Alert fired
Jira	`#[project]`	Issue moved to Done
UptimeRobot	`#incidents`	Site down

Communication Norms

```
Response time expectations:
  #incidents:        < 5 minutes during business hours
  #[project]:        < 2 hours during business hours
  DMs:               < 4 hours during business hours
  #general:          < 24 hours

Thread rule:
  All replies to a message go in thread — do not pollute main channel

Decision rule:
  Decisions made in Slack must be documented in Confluence within 24h
```

A5. Git-Based Workflows

Branch Strategy (GitFlow-based)

```
main          ← Production. Protected. No direct commits. Tag per release.
develop       ← Integration branch. Protected. Auto-deploys to staging.
feature/*     ← Feature development. PR → develop.
fix/*         ← Bug fixes. PR → develop (or main for hotfix).
release/*     ← Release preparation. PR → main + develop.
hotfix/*      ← Emergency production fix. PR → main + develop.
```

Branch Naming Convention

```
feature/PROJ-123-short-description
fix/PROJ-456-fix-login-redirect
hotfix/PROJ-789-critical-auth-bypass
release/v1.3.0
chore/update-dependencies
docs/update-api-readme
```

Commit Message Convention

```
# Format: <type>(<scope>): <subject>
# Line 1: max 72 chars
# Line 3+: optional body

feat(auth): add password reset flow
fix(users): correct pagination offset on user list
docs(api): update Swagger tags for /orders endpoint
chore(deps): upgrade nestjs to v10.3.0
refactor(billing): extract invoice service from order module
test(auth): add integration test for token refresh
ci(deploy): add staging auto-deploy on develop push
perf(db): add index on orders.created_at for report queries

# Breaking change:
feat(api)!: rename /users endpoint to /accounts

BREAKING CHANGE: All clients must update API path from /users to /accounts
```

Pull Request Rules

```
PR title format:
  [PROJ-123] feat: Add password reset email flow

PR description template:
  ## What
  [Brief description of the change]

  ## Why
  [Business context / linked issue]

  ## How to test
  1. Log in as user@test.com
  2. Click "Forgot password"
  3. Check email arrives within 30 seconds
  4. Complete reset flow

  ## Screenshots (if UI change)
  [Attach before/after]

  ## Checklist
  - [ ] Tests added/updated
  - [ ] Docs updated
  - [ ] No secrets committed
  - [ ] Migration included (if schema change)

Merge rules:
  - Minimum 1 approval
  - CI must pass
  - No unresolved comments
  - Squash merge to main/develop
```

A6. Sprint Planning & Backlog Refinement

Backlog Refinement (Grooming)

```
Frequency: Mid-sprint (Thursday of Week 1)
Duration: 60 minutes max
Participants: PO, PM, TL, 1–2 engineers

Agenda:
  1. PO presents top 10 upcoming stories (20 min)
  2. Engineers ask clarifying questions (20 min)
  3. Engineers estimate pointed stories (15 min)
  4. Acceptance criteria reviewed and adjusted (5 min)

Output:
  - Stories are estimated and "Ready" for sprint planning
  - PO has updated priority order in backlog
```

Sprint Planning

```
Frequency: First Monday of each sprint
Duration: 2 hours max
Participants: Full team

Agenda:
  1. PM shares sprint goal (5 min)
  2. PO presents top stories from backlog (10 min)
  3. Team commits to sprint capacity (10 min)
     Capacity = available_days × 6 hours × engineer_count × 0.7
  4. Pull stories into sprint until capacity filled (20 min)
  5. Stories assigned to engineers (10 min)
  6. Identify dependencies and risks (10 min)
  7. Confirm sprint goal (5 min)

Output:
  - Sprint backlog populated in Jira
  - Engineers know their assignments
  - Sprint goal stated in Jira sprint description
```

A7. Release Planning

Release Types in Jira

```
Minor release (vX.Y.0):
  - End of one or more development sprints
  - Full UAT required
  - Full staging validation

Patch release (vX.Y.Z):
  - Bug fixes only
  - Abbreviated UAT
  - Same-day deployment approved by TL

Major release (vX.0.0):
  - Breaking API changes
  - Migration guide required
  - Extended UAT window (1–2 weeks)
  - Client sign-off mandatory
```

Release Tracking in Jira

```
Jira Releases (Versions):
  - Create a version: v1.3.0
  - Set release date
  - Link all stories/bugs to Fix Version = v1.3.0
  - Track progress via Version report
  - Release when all stories Done and tested
  - Jira marks version as Released — generates release notes

GitHub side:
  - Tag: git tag v1.3.0
  - GitHub Release: auto-generate from tag + changelog
```

A8. QA & Acceptance Workflow

```
QA Testing flow in Jira:

Story moved to Testing by developer
    ↓
QA assigns themselves
    ↓
QA tests against acceptance criteria
    ↓
    ├── PASS → Move to Staging / UAT
    └── FAIL → Create bug linked to story → Move story back to In Progress
                    ↓
               Bug fixed → Move back to Testing
                    ↓
               QA re-tests

UAT flow (Staging):
  PO / client tests on staging
    ↓
    ├── ACCEPTED → Story moved to Done
    └── REJECTED → Comment added → Story reopened → Back to In Progress
```

Test Case Tracking (Confluence or Jira Xray)

```
For client projects with formal QA:
  - Use Jira Xray plugin for test management
  - Test Plans linked to versions
  - Test Cycles per sprint

For internal projects:
  - Test cases documented in Confluence
  - QA notes added to Jira issue comments
```

A9. Incident Management

Incident in Jira

```
Issue type: Incident (custom type)
Fields:
  - Severity: SEV1 / SEV2 / SEV3 / SEV4
  - Affected component
  - Detection time
  - Resolution time
  - Root cause
  - Link to post-mortem (Confluence page)

Workflow:
  Open → Investigating → Mitigating → Resolved → Post-mortem → Closed
```

Incident Slack Workflow

1. Alert fires → #incidents channel
2. IC posts: "🔴 SEV[N] INCIDENT: [description]. IC: [Name]. Bridge: [Slack thread]"
3. All investigation in thread
4. IC posts updates every 15–30 min
5. IC posts: "✅ RESOLVED: [time]. Root cause: [brief]. Post-mortem: [link]"
6. Post-mortem written in Confluence within 48h
7. Jira incident ticket closed with post-mortem link

A10. Change Requests

Change Request in Jira

```
Issue type: Change Request
Required fields:
  - Description
  - Business justification
  - Technical impact (estimated by TL)
  - Effort estimate (story points)
  - Risk level: Low / Medium / High
  - Priority

Workflow:
  New → Under Review → Approved → Scheduled → In Progress → Done
          ↓
      Rejected → Closed with reason
```

Case B - Odoo + GitHub Projects

B1. Tool Stack

Tool	Purpose	Cost
Odoo Community	Project management, client portal, CRM, timesheets	Free (self-hosted)
GitHub Projects	Engineering kanban board, sprint view	Free (with GitHub)
GitHub Issues	Bugs, tasks, feature requests	Free
GitHub Milestones	Release tracking	Free
GitHub Pull Requests	Code review, merge workflow	Free
GitHub Actions	CI/CD	Free (2,000 min/mo)
GitHub Discussions	Team async communication	Free

Tool	Purpose	Cost
Slack (optional)	Notifications only (GitHub → Slack)	Free tier

Total tooling cost: €0 (using Odoo Community + GitHub Free/Pro)

B2. Odoo for Business Project Management

Odoo Project Module Configuration

```
Odoo → Project → Configuration:

Settings to enable:
  ✅ Stages (customize per project)
  ✅ Task dependencies
  ✅ Subtasks
  ✅ Timesheets (if billing by time)
  ✅ Customer portal access (for client visibility)
  ✅ Milestones
  ✅ Ratings (client satisfaction)
  ✅ Planned dates

Create a project per client or product:
  Project: LIONGATE ERP
  Project: Client Alpha – Web Platform
  Project: Internal Dashboard
```

Odoo Task Structure

```
Odoo Project Task hierarchy:

Epic (represented as a Task with subtasks or as a Project Stage)
└── Task (main unit of work)
    ├── Subtask 1
    └── Subtask 2

Alternatively:
  Use Project Tags to simulate epics:
  Tags: [Epic: Authentication] [Epic: Dashboard] [Epic: Reports]
```

Odoo Project Stages (per project)

```
Configure per project in Odoo:

Backlog → Ready → In Progress → In Review → Testing → Staging → Done → Cancelled

Map to GitHub:
  Backlog       → GitHub Issue: open, no milestone
  Ready         → GitHub Issue: open, milestone assigned
  In Progress   → GitHub Issue: assigned to developer
  In Review     → GitHub PR: open and linked to issue
  Testing       → GitHub Issue: label "in-testing"
  Staging       → GitHub Issue: label "in-staging"
  Done          → GitHub Issue: closed
```

Odoo Customer Portal for Clients

```
Odoo → Project → Project → Customer Portal

Enable portal access:
  Settings → Users → Portal (client receives email invitation)

What clients see in the portal:
  - Their project tasks (read-only or with commenting)
  - Milestones and progress
  - Timesheets (if shared)
  - Documents uploaded to tasks

This replaces Confluence for client-facing documentation.
```

Odoo Timesheets Integration

```
If LIONGATE bills by time:

  Engineer logs time in Odoo Timesheet against each task
  PM reviews weekly timesheet summary
  Odoo generates invoice from timesheets (Odoo Invoicing module)
  Client receives invoice via Odoo email or portal

No additional billing tool required.
```

Odoo CRM → Project Flow

```
CRM Opportunity → Won → Create Project automatically
  ↓
  New project created with:
    - Client linked
    - Sales order linked (if applicable)
    - Project template applied (optional)
  ↓
  PM assigns tasks in project
  ↓
  Engineers work in GitHub
  ↓
  PM tracks in Odoo
```

B3. GitHub Projects for Engineering Execution

GitHub Projects v2 Setup

```
Location: github.com/orgs/liongate/projects (org-level)
or: github.com/liongate/repo/projects (repo-level)

Use org-level projects for cross-repo views.
Use repo-level projects for single-product teams.

Create one project per active product or client:
  "LIONGATE ERP - Engineering"
  "Client Alpha Platform - Engineering"
  "Internal Dashboard - Engineering"
```

GitHub Project Views

View	Type	Purpose
Sprint Board	Board	Current sprint Kanban
Backlog	Table	All items, sortable/filterable
Roadmap	Roadmap	Timeline view of milestones

View	Type	Purpose
By Assignee	Board	Work per engineer
By Epic	Board	Grouped by epic label

GitHub Project Fields (Custom)

```
Add these custom fields to the GitHub Project:

Field           Type          Values
──────────────────────────────────────────────────────
Sprint          Iteration     2-week iterations
Story Points    Number        1, 2, 3, 5, 8, 13
Priority        Single select Must / Should / Could / Won't
Type            Single select Story / Task / Bug / Tech Debt
Epic            Single select [List of epics]
Status          Single select Backlog / Ready / In Progress / In Review / Testing / Sta
Assignee        [Built-in]
Milestone       [Built-in]
```

B4. GitHub Issues & Pull Requests

Issue Templates

Create `.github/ISSUE_TEMPLATE/` directory in each repo:

`.github/ISSUE_TEMPLATE/user-story.md`

```
---
name: User Story
about: Feature or user-facing requirement
labels: story
---

## User Story

As a **[role]**,
I want to **[action]**,
So that **[business value]**.

## Acceptance Criteria

- [ ] GIVEN [precondition] WHEN [action] THEN [outcome]
- [ ] GIVEN [precondition] WHEN [action] THEN [outcome]

## Technical Notes

[Optional: DB changes, API endpoints, dependencies]

## Odoo Task Reference

Odoo Task: [URL or ID]

## Estimate

Story Points: [ ]
```

.github/ISSUE_TEMPLATE/bug-report.md

```
---
name: Bug Report
about: Something is broken
labels: bug
---

## Summary

[Concise description]

## Severity

- [ ] P0 - Critical (production down)
- [ ] P1 - High (feature broken, no workaround)
- [ ] P2 - Medium (workaround exists)
- [ ] P3 - Low (cosmetic)

## Steps to Reproduce

1.
2.
3.

## Expected Result

[What should happen]

## Actual Result

[What happens instead]

## Environment

- Version/commit:
- Browser/OS:

## Evidence

[Screenshots, logs]

## Odoo Task Reference
```

```
Odoo Task: [URL or ID]
```

.github/ISSUE_TEMPLATE/tech-debt.md

```
---
name: Technical Debt
about: Internal improvement or refactoring
labels: tech-debt
---

## Description

[What needs to be improved and why]

## Impact if not addressed

[What degrades without this fix]

## Proposed solution

[How to fix it]

## Effort estimate

Story Points: [ ]
```

.github/ISSUE_TEMPLATE/incident.md

```
---
name: Incident
about: Production incident report
labels: incident, P0
---

## Incident Summary

**Severity:** SEV[ ]
**Detection time:** [Time]
**Resolution time:** [Time or "Ongoing"]
**Affected service:** [Service name]

## Timeline

- [HH:MM] - [Event]
- [HH:MM] - [Event]

## Root Cause

[Description]

## Resolution

[What was done]

## Action Items

- [ ] [Action] - @[owner]

## Post-Mortem

[Link to post-mortem document]
```

Pull Request Template

.github/pull_request_template.md

```
## Summary

[What this PR does]

## Related Issue

Closes #[issue-number]
Odoo Task: [URL]

## Type of Change

- [ ] New feature
- [ ] Bug fix
- [ ] Refactor
- [ ] Tech debt
- [ ] Documentation
- [ ] CI/CD change

## How to Test

1. [Step 1]
2. [Step 2]

## Screenshots (if UI change)

| Before | After |
| ------ | ----- |
|        |       |

## Checklist

- [ ] Tests added/updated and passing
- [ ] Documentation updated if needed
- [ ] Migration included (if DB changes)
- [ ] No hardcoded secrets
- [ ] PR linked to GitHub issue
- [ ] Odoo task updated to "In Review"
```

B5. Milestones

GitHub Milestones = Releases

```
Milestones represent releases or major delivery checkpoints.

Create milestones:
  v1.0.0 – Initial Launch        Due: 2024-03-01
  v1.1.0 – Post-launch fixes     Due: 2024-04-01
  v1.2.0 – Dashboard feature     Due: 2024-05-01

Rules:
  – Every issue in a sprint is assigned to the correct milestone
  – Milestone closes when all issues are closed
  – GitHub shows % completion automatically

Link milestone to Odoo:
  – Odoo Project Milestone = GitHub Milestone
  – PM tracks in Odoo, engineers track in GitHub
  – PM manually syncs status weekly (or via Odoo webhook → GitHub API)
```

Milestone Progress View

```
GitHub → Project → Milestones tab:

v1.2.0 Dashboard feature
███████████░░░ 75% – 15/20 issues closed
Due: 2024-05-01 – 12 days remaining

Open issues: 5
Closed issues: 15
```

B6. Labels

Standard Label Set (apply to all repos)

```
# Create these labels in every GitHub repo
# Use GitHub CLI: gh label create "name" --color "COLOR" --description "DESC"

# Type labels
gh label create "story"      --color "0075ca" --description "User story"
gh label create "task"       --color "e4e669" --description "Technical task"
gh label create "bug"        --color "d73a4a" --description "Something is broken"
gh label create "tech-debt"  --color "fef2c0" --description "Internal improvement"
gh label create "incident"   --color "b60205" --description "Production incident"
gh label create "docs"       --color "cfd3d7" --description "Documentation"

# Priority labels
gh label create "P0-critical" --color "b60205" --description "Production down"
gh label create "P1-high"     --color "d93f0b" --description "Core feature broken"
gh label create "P2-medium"   --color "e4e669" --description "Workaround exists"
gh label create "P3-low"      --color "0e8a16" --description "Minor issue"

# Layer labels
gh label create "backend"    --color "1d76db" --description "Backend changes"
gh label create "frontend"   --color "0052cc" --description "Frontend changes"
gh label create "devops"     --color "5319e7" --description "Infrastructure/CI"
gh label create "database"   --color "006b75" --description "DB schema or query"

# Status labels (complement GitHub Project status field)
gh label create "in-review"    --color "fbca04" --description "PR open, under review"
gh label create "in-testing"   --color "f9d0c4" --description "QA is testing"
gh label create "in-staging"   --color "c2e0c6" --description "Deployed to staging"
gh label create "blocked"      --color "e11d48" --description "Blocked by dependency"
gh label create "needs-info"   --color "e4e669" --description "Waiting for clarificatio

# Epic labels (one per epic)
gh label create "epic:auth"       --color "8250df" --description "Epic: Authentication"
gh label create "epic:dashboard"  --color "8250df" --description "Epic: Dashboard"
gh label create "epic:api"        --color "8250df" --description "Epic: API"
```

B7. Boards & Views

Sprint Board Configuration

```
GitHub Project → Board view → Group by: Status

Columns:
  Backlog | Ready | In Progress | In Review | Testing | Staging | Done

Rules:
  - Backlog: Issues without sprint assigned
  - Ready: Issues with sprint assigned, estimated, acceptance criteria complete
  - In Progress: Developer assigned and working
  - In Review: PR linked and open
  - Testing: QA actively testing
  - Staging: Deployed to staging, awaiting UAT
  - Done: Closed and accepted

WIP Limits (manual enforcement):
  In Progress: max 2 issues per engineer
  In Review: max 3 open PRs per engineer
```

Backlog Table View

```
GitHub Project → Table view

Columns visible:
  Title | Type | Priority | Assignee | Story Points | Sprint | Milestone | Status

Sort by: Priority (default)
Filter: Sprint = "Current Sprint" (for sprint view)
Filter: Sprint = none (for backlog grooming)
```

Roadmap View

```
GitHub Project → Roadmap view

Shows: Milestones as date ranges
Items grouped by: Milestone / Epic
Useful for: PM and client roadmap discussions
```

B8. Workflows & Automation

GitHub Actions for Project Automation

Auto-assign issues to project when created:

```yaml
# .github/workflows/project-automation.yml
name: Project Automation

on:
  issues:
    types: [opened, labeled]
  pull_request:
    types: [opened, ready_for_review, closed]

jobs:
  add-to-project:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/add-to-project@v0.5.0
        with:
          project-url: https://github.com/orgs/liongate/projects/1
          github-token: ${{ secrets.PROJECT_TOKEN }}
```

Auto-move issue to "In Review" when PR is opened:

```yaml
move-to-review:
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request' && github.event.action == 'opened'
  steps:
    - name: Extract issue number from PR body
      id: extract
      run: |
        ISSUE=$(echo "${{ github.event.pull_request.body }}" | grep -oP '(?<=Closes #)\
        echo "issue=$ISSUE" >> $GITHUB_OUTPUT
    - name: Add label to issue
      if: steps.extract.outputs.issue != ''
      uses: actions/github-script@v7
      with:
        script: |
          github.rest.issues.addLabels({
            owner: context.repo.owner,
            repo: context.repo.repo,
            issue_number: ${{ steps.extract.outputs.issue }},
            labels: ['in-review']
          })
```

Auto-close issue when PR is merged:

```yaml
close-issue-on-merge:
  runs-on: ubuntu-latest
  if: github.event.pull_request.merged == true
  steps:
    - name: Comment and close linked issue
      uses: actions/github-script@v7
      with:
        script: |
          // GitHub auto-closes issues referenced with "Closes #N" - this is built-in
          // Just add a comment with deploy target
          const pr = context.payload.pull_request;
          console.log(`PR merged: ${pr.title}`);
```

Auto-label by file path:

```yaml
# .github/labeler.yml
backend:
  - changed-files:
      - any-glob-to-any-file: "src/**"
      - any-glob-to-any-file: "api/**"

frontend:
  - changed-files:
      - any-glob-to-any-file: "frontend/**"
      - any-glob-to-any-file: "web/**"

devops:
  - changed-files:
      - any-glob-to-any-file: ".github/**"
      - any-glob-to-any-file: "docker-compose*"
      - any-glob-to-any-file: "Dockerfile*"
      - any-glob-to-any-file: "infrastructure/**"

database:
  - changed-files:
      - any-glob-to-any-file: "src/database/migrations/**"


# .github/workflows/labeler.yml
name: Label PR by file path
on: [pull_request]
jobs:
  label:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/labeler@v5
        with:
          repo-token: ${{ secrets.GITHUB_TOKEN }}
```

B9. Approval Flow

Code Review Approval

```
CODEOWNERS file: .github/CODEOWNERS

# Require Tech Lead review for:
/src/database/migrations/  @liongate/tech-lead
/infrastructure/            @liongate/devops
/.github/workflows/         @liongate/devops
/src/auth/                  @liongate/tech-lead

# General code: any 1 reviewer
*                           @liongate/engineers

Branch protection rules (GitHub Settings → Branches):
  Branch: main
    ✅ Require pull request before merging
    ✅ Require approvals: 1
    ✅ Dismiss stale pull request approvals on new commits
    ✅ Require review from Code Owners
    ✅ Require status checks to pass (CI)
    ✅ Require branches to be up to date
    ✅ Do not allow bypassing the above settings

  Branch: develop
    ✅ Require pull request before merging
    ✅ Require approvals: 1
    ✅ Require status checks to pass (CI)
```

Deployment Approval (GitHub Environments)

```
GitHub → Settings → Environments

Environment: staging
  Protection rules: none (auto-deploy on develop merge)
  Secrets: STAGING_HOST, STAGING_SSH_KEY

Environment: production
  Protection rules:
    ✅ Required reviewers: @liongate/tech-lead, @liongate/pm
  Wait timer: 0 minutes
  Secrets: PROD_HOST, PROD_SSH_KEY

Effect:
  Merge to main → CI builds → Deployment to production PAUSES
  → Slack notification: "Production deployment waiting for approval"
  → Reviewer approves in GitHub UI
  → Deployment proceeds
```

Odoo Approval for Business Tasks

```
Odoo → Project → Task → Stage: "Staging"

Configure stage approval in Odoo:
  - Task reaches "Staging" stage → PM notified
  - PM reviews on staging environment
  - PM approves → moves task to "Done"
  - PM rejects → moves task back to "In Progress" with comment

This separates:
  Engineering approval = GitHub PR review
  Business approval = Odoo task stage move by PM/PO
```

B10. Release Tracking

Release Workflow (Odoo + GitHub)

```
Phase 1 — Sprint completion
  GitHub: All sprint issues closed
  GitHub Milestone: 100% closed
  Odoo: Milestone marked complete

Phase 2 — Release preparation
  GitHub: Create release branch: release/v1.3.0
  GitHub: Final staging deployment
  Odoo: PM confirms staging sign-off

Phase 3 — Tagging
  git tag v1.3.0 -m "Release v1.3.0"
  git push origin v1.3.0

Phase 4 — GitHub Release
  GitHub → Releases → Draft new release
  Tag: v1.3.0
  Title: v1.3.0 — [Feature summary]
  Generate release notes (auto from merged PRs)
  Publish release

Phase 5 — Production deployment
  Push to main → CI → GitHub Environment approval → Deploy

Phase 6 — Post-release
  Odoo: Update project milestone to "Released"
  Odoo: Log release in project notes
  Notify client via Odoo portal or email
```

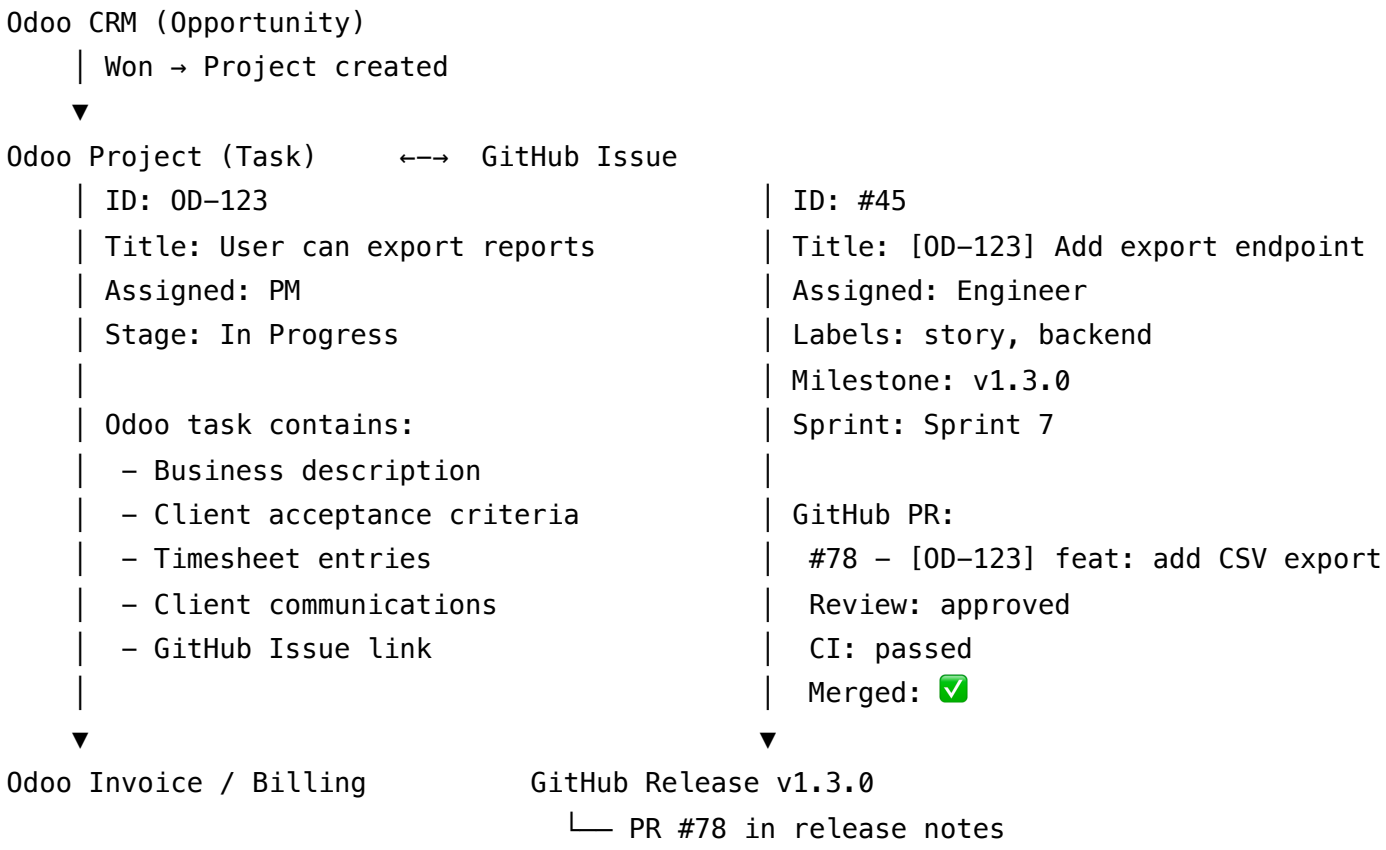
Release Notes Auto-Generation

```yaml
# .github/release.yml
changelog:
  categories:
    - title: "🚀 New Features"
      labels: ["story", "enhancement"]
    - title: "🦠 Bug Fixes"
      labels: ["bug"]
    - title: "🔧 Tech Debt & Refactoring"
      labels: ["tech-debt", "refactor"]
    - title: "📦 DevOps & Infrastructure"
      labels: ["devops"]
    - title: "📝 Documentation"
      labels: ["docs"]
```

This auto-generates release notes from PR titles grouped by label.

B11. Traceability: Business ↔ Engineering

Traceability Map



Linking Convention

Every GitHub Issue must reference its Odoo task in the body:

```
## Odoo Task Reference

Odoo Task: https://erp.liongate.com/odoo/project/task/123
```

Every Odoo task must reference its GitHub Issue:

```
Odoo Task → Description → Add:
"Engineering: https://github.com/liongate/project/issues/45"
```

Traceability Table (maintained by PM weekly)

```
| Odoo Task | GitHub Issue | PR   | Status      | Milestone |
| --------- | ------------ | ---- | ----------- | --------- |
| OD-123    | #45          | #78  | Done        | v1.3.0    |
| OD-124    | #46          | #79  | In Review   | v1.3.0    |
| OD-125    | #47          | -    | In Progress | v1.3.0    |
```

Detailed Comparison: Case A vs Case B

Feature	Case A (Jira+Confluence)	Case B (Odoo+GitHub)
Issue tracking	Jira Software	GitHub Issues
Board / Kanban	Jira Board	GitHub Projects
Documentation	Confluence wiki	GitHub Wiki / Repo docs
Sprint management	Jira Sprints	GitHub Project Iterations
Roadmap	Jira Roadmap	GitHub Project Roadmap
Release tracking	Jira Versions	GitHub Milestones + Releases
PR integration	Via Jira-GitHub plugin	Native GitHub
Business management	Jira (limited)	Odoo (CRM, invoicing, portal)
Time tracking	Tempo (paid plugin)	Odoo Timesheets (free)
Client portal	Jira Service Desk (paid)	Odoo Customer Portal (free)
Invoicing	External tool	Odoo Invoicing
Incident management	Jira incident type	GitHub Issue + post-mortem
Dependency tracking	Jira issue links	GitHub linked issues
Automation	Jira Automation	GitHub Actions
Mobile access	Jira mobile app	GitHub mobile app
Cost (team of 5)	~€150–€250/mo	~€0–€20/mo

Feature	Case A (Jira+Confluence)	Case B (Odoo+GitHub)
Setup complexity	High	Low
Team learning curve	High (Jira is complex)	Medium
Industry recognition	Very high (CV-worthy)	High (GitHub is universal)
Audit trail	Jira issue history	GitHub issue + PR history
SLA tracking	Jira Service Management	Odoo (basic)
OKR / Goal tracking	Jira Goals (new)	Manual in Odoo
Scalability	Very high (enterprise)	High

When to Use Each Approach

Use Case A (Jira + Confluence + Slack) when:

Scenario	Reason
Client mandates Jira	Contractual / client policy
Team > 15 engineers	Jira scales better for large teams
Complex multi-project dependencies	Jira portfolio management
Formal SLA tracking required	Jira Service Management
Regulatory compliance (ISO, SOC2)	Jira provides required audit trails
Client has existing Atlassian suite	Easier integration
Hiring senior engineers expecting Jira	Industry standard familiarity

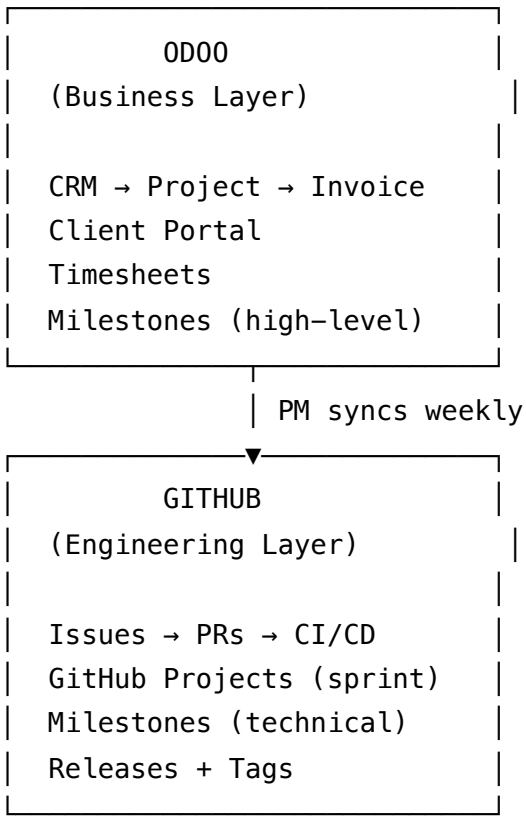
Use Case B (Odoo + GitHub) when:

Scenario	Reason
LIONGATE internal projects	Full control, zero cost

Scenario	Reason
Small to medium client projects	Lean, fast, effective
Client does not have tool requirements	Default LIONGATE approach
Budget-sensitive projects	€0 tooling cost
Business + engineering in one place	Odoo handles both
Team < 15 engineers	GitHub Projects scales well here
Client wants portal access	Odoo Customer Portal is free
Billing and project in one system	Odoo handles invoicing too

Hybrid Model: LIONGATE Recommended Approach

For most projects, LIONGATE SARL should operate a **hybrid model**:



Layer Responsibilities

Layer	Tool	Managed by	Contains
Business	Odoo	PM / PO	Client requirements, timesheets, invoicing, milestones, client comms
Engineering	GitHub	TL / Engineers	Code, issues, PRs, CI/CD, releases, tech decisions

Sync Protocol

```
Weekly PM sync (30 min — PM performs):
  1. Review GitHub Project board
  2. Update corresponding Odoo tasks with current status
  3. Update Odoo milestone progress %
  4. Flag blocked items for stand-up

Release sync (per release — PM + TL):
  1. GitHub milestone closed
  2. Odoo milestone marked "Done"
  3. GitHub Release published
  4. Odoo: notify client via portal or email
  5. Odoo: log release in task/project notes
```

Onboarding Checklist per Model

Case A - Jira + Confluence Setup

- [ ] Jira Cloud workspace created
- [ ] Projects created per product/client
- [ ] Issue types configured (Story, Task, Bug, Incident, CR)
- [ ] Workflow configured per project
- [ ] Custom fields configured (Story Points, Priority, Epic)
- [ ] Sprint boards created
- [ ] GitHub-Jira integration installed and connected
- [ ] Confluence space created per project

- [ ] Page templates created (Meeting Notes, ADR, Runbook)
- [ ] Slack connected to Jira (notifications)
- [ ] Slack connected to GitHub (PR and CI notifications)
- [ ] Team members invited to Jira + Confluence
- [ ] CODEOWNERS file in repos
- [ ] Branch protection rules configured in GitHub

Case B - Odoo + GitHub Setup

GitHub side:

- [ ] GitHub Organization created: `github.com/liongate`
- [ ] Repositories created per project
- [ ] Issue templates created (`.github/ISSUE_TEMPLATE/`)
- [ ] PR template created (`.github/pull_request_template.md`)
- [ ] Labels created (all standard labels per label list above)
- [ ] GitHub Project created per product
- [ ] Project custom fields configured (Sprint, Points, Priority, Type, Epic)
- [ ] Milestones created per release
- [ ] Branch protection rules configured
- [ ] CODEOWNERS file created
- [ ] GitHub Actions workflows: CI, labeler, project automation
- [ ] GitHub Environments: staging + production (with approval gate)
- [ ] `release.yml` configured for auto release notes

Odoo side:

- [ ] Odoo Project module enabled
- [ ] Projects created (one per client/product)
- [ ] Project stages configured per project
- [ ] Milestones configured in Odoo
- [ ] Client users invited to Odoo portal
- [ ] Timesheets module enabled (if billing by time)
- [ ] CRM → Project link configured
- [ ] Odoo email/SMTP configured for notifications

Sync setup:

- [ ] Linking convention communicated to team
- [ ] PM weekly sync process documented and scheduled

- [ ] Traceability table template set up

Templates Library

GitHub Project Sprint Template

```
Sprint [N] — [Start Date] to [End Date]

Sprint Goal:
  [One sentence describing what this sprint delivers]

Capacity:
  Engineers: [N]
  Working days: 10
  Available hours: [N × 10 × 6 × 0.7]
  Story points: [N]

Committed items:
  [List of issue links]

Carry-over from previous sprint:
  [List if any]

Blockers / dependencies:
  [List if any]
```

Odoo Weekly Status Report Template

```
📊 Project Status — [Project Name]
Week of [Date]

🟢 On Track / 🟡 At Risk / 🔴 Behind

Progress:
  Sprint [N]: [X/Y] issues closed ([Z]%)
  Milestone [v1.X]: [X/Y] issues closed ([Z]%)

Completed this week:
  - [Item 1] — [Engineer]
  - [Item 2] — [Engineer]

In Progress:
  - [Item 3] — [Engineer] — ETA: [Date]

Blockers:
  - [Blocker] — Owner: [Name] — Expected resolution: [Date]

Next week:
  - [Planned item 1]
  - [Planned item 2]

Time logged this week: [X] hours
Budget consumed: [X]% of total
```

GitHub Release Announcement Template

```
# Release v1.3.0 - [Feature Name]

**Release Date:** [Date]
**Deployed:** Production ✅

## What's New

### 🚀 Features

- [Feature 1] - allows users to [benefit]
- [Feature 2] - enables [workflow]

### 🐛 Bug Fixes

- Fixed: [Bug description] (#issue)
- Fixed: [Bug description] (#issue)

### ⚠️ Breaking Changes

None / [Describe if any]

### 📦 Infrastructure

- [Infrastructure change if any]

## How to Update

[Steps if any action required by users]

## Known Issues

[List if any, with workarounds]

Full changelog: [link to CHANGELOG.md]
```

Document owner: LIONGATE SARL Engineering & Project Management Team
Last updated: See Git history
Next review: Quarterly or when tool stack changes